COMP3334 Computer Systems Security

Group Project – Team89

Contribution Table

SU,Yi 22103815d	25%
TAI, Lee Na 22076162d	25%
TSANG, Man Ho 24028571d	25%
YEUNG,Hang 22027226d	25%

1. Abstract

In the digital information age, a secure online storage system is essential for users to effectively manage documents and reduce security risks. The project proposes a client application for file uploading, and a server application for monitoring its file management and user authentication. The system is built on the assumption of passive adversaries, including servers monitoring communications and unauthorized users trying to access legitimate accounts.

Key security features include user authentication through hashed passwords and time-based one-time passwords (TOTP), data confidentiality through client-side encryption, and integrity through strict access control measures. The architecture employs user management, data encryption, and log auditing algorithms to ensure accountability and traceability. The system is designed to protect user data from unauthorized access and maintain the integrity of the storage environment. The project highlights the importance of secure design in protecting sensitive information and providing basic document management capabilities.

2. Introduction

2.1 Objective

In a time where digital data is integral to our daily lives, online storage systems have become an essential tool for users to store, retrieve and share files across devices. While these systems provide us with convenience, it can pose substantial security risks as they manage sensitive information vulnerable to unauthorized access most likely in an unknown way. This project aims to design and implement a secure online storage system that focuses on data protection.

2.2 Assumptions

Before the implementation of our secure system, several assumptions are clarified to guide the design and development process:

1. Adversaries are passive and do not conduct active attacks, such as man-in-the-middle interventions
2. The server is inaccessible to encryption keys and cannot decrypt user files even with full access to stored data
3. Admin account is set up using a secure database configuration to reduce risks from SQL command execution

2.3 Security features

Our primary aim is to create a platform that protects user data while delivering essential file storage system features. Here are the security requirements implemented to protect against potential threats:

2.3.1 User Authentication

System verifies user identities through hashed passwords and One-Time Password (OTP) to prevent unauthorized account access

2.3.2 Confidentiality

System ensures that files uploaded to the server remain inaccessible in plaintext (including the server itself) to unauthorized entities by encrypting them on the client side with securely generated keys stored locally

2.3.3 Integrity

System ensures access control, restricting users to read and modify only their own files or files shared with them by other users.

2.3.4 Availability

System ensures files are stored and made available for user to read, write, update and delete when it is requested.

2.3.5 Non-Repudiation

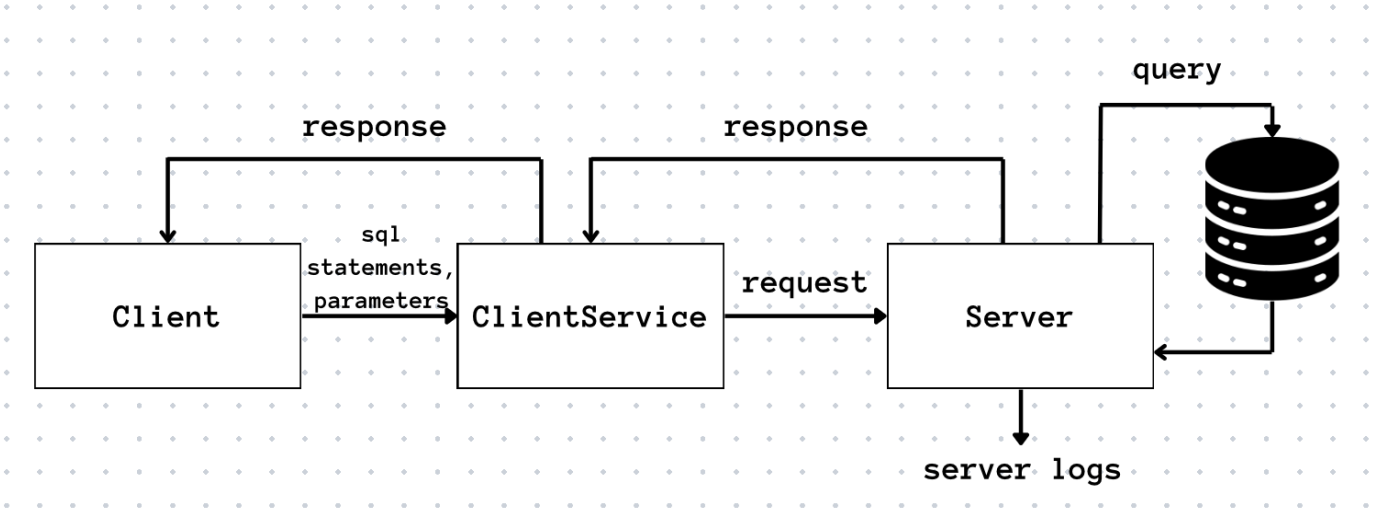
System records the critical operations by audit trace logs—such as login, logout, uploads, deletions, and sharing that can be accessed by admin only, ensuring users cannot deny their actions

2.3.6 Security to general threats

System defends against common vulnerabilities, including SQL injections and malicious file name exploits (e.g., "../file.txt") to maintain integrity and safety

2.4 System Design

The system follows a client-server architecture, where the Client and Server components communicate over a network using `java.net.Socket` at `localhost:12345`



Client-Side:

- Client.java
- ClientService.java
- **uploads/**: Stores files the user wants to upload.
- **downloads/**: Stores decrypted files after downloading.
- **encryption_key.key**: Holds the AES key for file encryption/decryption, kept on the client side for security

Server-Side:

- Server.java

3. Threat Models

Our online storage system identifies two primary threat models involving passive adversaries that pose risks to the security:

Adversary 1: The Server Machine	
Who	The machine hosting the server.java program
Abilities	• Monitors all network communications between the Server and Client programs, including server logs such as **LOGIN**, **REGISTER**, **UPLOADFILE**, **DOWNLOADFILE**, and others transmitted via **ObjectInputStream**. • Accesses all data stored on the database, including encrypted file data from **Comp3334_AccFile** table, hashed passwords in the **Comp3334_CSSAccount** table, and audit logs in the **Comp3334_LogCSS** table within the Oracle database
Objective	To decrypt and access the plaintext content of users' uploaded files stored in the **fileData** column of the **Comp3334_AccFile** table
Constraints	Served as a passive adversary, it does not modify messages, forge responses, or perform active attacks

Adversary 2: Unauthorized User	
Who	Unauthorized individual that uses a legitimate user's computer running client.java
Abilities	• Access to register/login interface via console • Attempts to bypass authentication by exploiting weak passwords or intercepting the TOTP stored temporarily in phone.txt before it expires • Accesses local files on the user's machine, such as the phone.txt file containing the Time-Based One-Time Password (TOTP) • Observes client-side operations, including file uploads from the uploads folder, downloads to the downloads folder
Objective	1. To gain unauthorized access to a user's account and retrieve their encrypted files from the **Comp3334_AccFile** table,

	2. decrypt them using the locally stored **SecretKey**, and potentially manipulate or share those files
Constraints	Served as a passive adversary, it does not modify client codes, carry out man-in-the-middle attack, or injecting malicious inputs. The attacks are limited to exploit existing data and credentials

3. Algorithms you designed to implement functionalities

3.1 User Management:

3.1.1 Username must be unique

To ensure that usernames are unique during user registration, the client.java will check if the provided username or email is already registered in the database (See the SQL queries below):

```
SELECT accNo FROM Comp3334_CSSAccount WHERE email = ?
```

```
SELECT accNo FROM Comp3334_CSSAccount WHERE loginName = ?
```

If the username or email is already in use, the system will prevent the account from being registered and display a message such as: “This account name has already been registered.”

Additionally, usernames must not contain any of the following characters `"',!%-+`~><$/\|&*()[]{};:?#^`. The input restriction here is to prevent SQL injection issues and ensure the integrity of the database.

3.2 Password Hashing:

To securely store user passwords, we have decided to use PBKDF2 (Password-Based Key Derivation Function 2) with HMAC-SHA256 to ensure the password are not exposed in database by storing in non-plaintext. The following is the password hashing procedure:

1. Generate a random salt by `java.security.SecureRandom`
2. Hash the password with the salt using PBKDF2, producing string e.g. 'salt:hashed' for storage
3. Verify by rehashing the input password with the stored salt and comparing it to the stored hash.

3.2.1 Salt Generation

128-bit random salt is generated by `java.security.SecureRandom`, as it is Java's a cryptographically secure pseudo-random number generator (CSPRNG) that ensures randomness. As for salt, which always generate a random salt for any users, even if there are two users using the same password as "x0", the hashed passwords are still different, for example <'salt:hashed'>:

`'AAAAAAAAAAAAAAAA:4b2a26ef18dc18e1094db03c9595d615'` and

`'BBBBBBBBBBBBBBBB:7b998d7fc375f627b4feeabb4168b071'`

3.2.2 PBKDF2 with HMAC-SHA256 (algorithm from `javax.crypto.SecretKeyFactory`)

We have designed to use PBKDF2 with HMAC-SHA256 as it is resistant to brute-force and pre-computed attacks (Kaliski, 2000). We configure the algorithm with 65,536 iterations and a 128-bit key length, this is because 65,536 iterations can significantly increase the computational cost of hashing, where it is effective to slows down brute-force guesses as they require a certain level of processing time (Menezes et al., 2018). As in research, PBKDF2 with its iterative nature can make parallelization on GPUs or ASICs less efficient compared to faster algorithms like SHA-1 (Menezes et al., 2018). HMAC-SHA256 is utilized as the pseudorandom function as unlike MAC, HMAC can resist vulnerabilities like length extension attacks as learnt from lectures, increasing the strength of cryptography.

3.2.3 Password verification

As for verification, the system will use the stored salt with the input password and pass it through the hash function. If the resulting hash matches the stored hashed password, then it passes the verification. User’s Hashed password will also be utilized as a part of the TOTP secret.

(See below for entire algorithm in our codes)

```
    // gen salt and use salt to hash function
    String salt = generateSalt();
    String hash = hashPasswordWithPBKDF2(password, salt);
    return salt + ":" + hash; // Store salt and hash together
}

public static String hashPasswordWithPBKDF2(String password, String salt) {
    try {
        KeySpec spec = new PBEKeySpec(password.toCharArray(), salt.getBytes(), ITERATIONS, KEY_LENGTH);
        SecretKeyFactory factory = SecretKeyFactory.getInstance(algorithm:"PBKDF2WithHmacSHA256");
        byte[] hash = factory.generateSecret(spec).getEncoded();
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1)
                hexString.append(c:'0');
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

public static String generateSalt() {
    SecureRandom random = new SecureRandom();
    byte[] salt = new byte[16]; // 128 bits
    random.nextBytes(salt);
    return new String(salt);
```

3.3 Data Encryption & Decryption:

3.3.1 AES-256-GCM

For data encryption and decryption, we chose the **Advanced Encryption Standard (AES)** with a 256-bit key operating in **Galois/Counter Mode (GCM)**. The decision was made after evaluating several encryption modes to ensure robust security for our project.

Initially, we noted that Java's `javax.crypto.Cipher` defaults to **Electronic Codebook (ECB) mode** for AES (Oracle, n.d.). However, ECB is insecure because as learnt by lectures, it encrypts identical plaintext blocks into identical ciphertext blocks when using the same key, making it vulnerable to frequency analysis and pattern recognition by passive adversaries.

Actually we have considered **Cipher Block Chaining (CBC) mode**, but while CBC prevents identical plaintext blocks from producing identical ciphertexts, it has drawbacks: If the same IV is reused with

the same key, CBC can leak information about plaintext. Ultimately, we selected **AES-256-GCM** because it overcomes all these limitations and provides a stronger security ability:

- **No IV Reuse Issue**: GCM ensures that each encryption uses a unique IV. So even if two identical plaintexts / file content are encrypted with the same key, different IVs result in different ciphertexts and prevents pattern analysis
- **Confidentiality**: Each block is encrypted with a unique counter value, making the ciphertext indistinguishable from random data and resistant to visible patterns (Dworkin, 2017)
- **Data Origin Authentication**: GCM generates a 128-bit authentication tag that verifies both the integrity and authenticity of the decrypted data, ensuring it originates from the owner who encrypted the file and not modified by others

3.3.2 Upload file & Encrypt (Client-side)

The file upload and encryption process is designed to ensure security and keeping the encryption key local to the client. The workflow is as follows:

1. A 256-bit AES key is generated or loaded using `loadOrGenerateKey()` and saved locally as **encryption_key.key** file
2. User places the file in the uploads folder and enters only the filename (e.g., "abcd.txt"), not the full path
3. A random 12-byte IV is generated using `SecureRandom` for each encryption

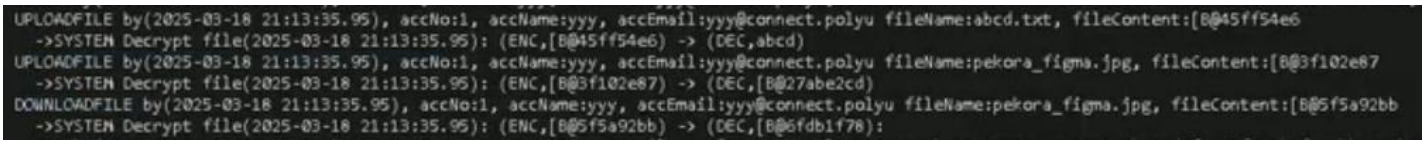
```
private static byte[] encryptFile(File file, SecretKey secretKey) {
    byte[] iv = generateIV(); // Generate a random IV
    try {
        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        GCMParameterSpec gcmSpec = new GCMParameterSpec(GCM_TAG_LENGTH * 8, iv);
        cipher.init(Cipher.ENCRYPT_MODE, secretKey, gcmSpec);

        byte[] fileData = Files.readAllBytes(file.toPath());
        byte[] encryptedData = cipher.doFinal(fileData);

        ByteBuffer byteBuffer = ByteBuffer.allocate(iv.length + encryptedData.length);
        byteBuffer.put(iv);
        byteBuffer.put(encryptedData);

        return byteBuffer.array();
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}
```

4. The file is encrypted with AES-256-GCM using the **encryption_key.key** and the IV implemented via `Cipher.getInstance("AES/GCM/NoPadding")`
5. Combine with the IV (prepended) and the 128-bit authentication tag, encrypted data is formed
6. The server only receives encrypted data not the secret key, and it stores the encrypted data in `Comp3334_AccFile` table from Oracle database

3.3.3 Server-Side Handling

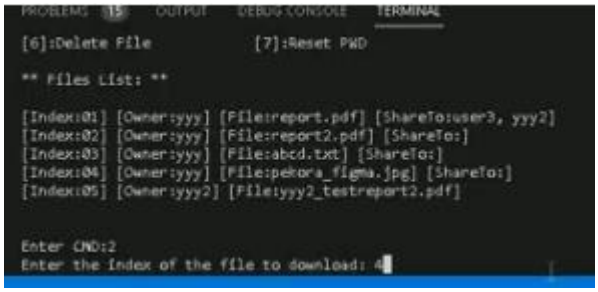
When the server receives an uploaded file:

- It reads the encrypted data and logs the action (e.g., "`UPLOADFILE`").
- It attempts to decrypt the data using its own **encryption_key.key**, which is generated or loaded independently on the server side. However, since this key differs from the client's key, decryption fails. It ensures the server cannot access the plaintext, maintaining confidentiality even if the server is compromised.
- The encrypted data is stored in the database, ensuring that server handles only ciphertext.



3.3.4 Download file & Decrypt (Client-side)

The file download and decryption process allows the client to retrieve and access their files securely, the idea is: only the client having the correct key can decrypt the file.



1. System uses index-based command-line interface (CLI). The user selects a file by entering its index number from a list of owned or shared files
2. Client requests encrypted file data from the server, retrieves it from the database if the user has permission (e.g., via `ClientService.decFile`)

```
// Retrieve the encrypted file from the database
try {
    System.out.println(x:"Retrieving file from database...");

    sql = "SELECT accNo, fileName, fileData FROM Comp3334_AccFile WHERE fileNo = ? AND (accNo = ? OR fileNo IN (SELECT fileNo FROM
    Comp3334_ShareFile WHERE accNo = ?))";
    resultList = ClientService.decFile(sql, selectedFileNo, acc.getAccNo()); // Get List<Map>

    if (resultList == null || resultList.isEmpty()) {
        System.out.println(x:"Error: File not found or access denied.");
        System.out.println(x:"Please press <enter> to continue...");
        console.readLine();
        break;
    }
```

3. For decryption, client extracts the IV from the first 12 bytes of the encrypted data

4. Using the local **encryption_key.key**, the remaining data (ciphertext and authentication tag) is decrypted with AES-256-GCM
5. Authentication tag is verified during decryption. If it fails (e.g., due to tampering), decryption aborts, ensuring data integrity
6. Decrypted file content is saved to the downloads folder for user access

3.3.5 Thought on preferring not to employ separate secret key (SK) approach

If we opt for this approach, each SK would need to be stored in the database. In such case, the SK must also be assigned to the shared users, allowing them to download the shared files. However, this project operates under the assumption that attackers cannot gain unauthorized access to the database to download files illegally. With this assumption, implementing a separate SK for each uploaded file will not enhance security significantly, instead, it will increase the resources required for database connections and management.

3.4 Access Control:

3.4.1 Theoretical Design

The access control mechanism ensures that:

a) Users can only add, edit and delete their own files
b) Users can share/unshare files with designated users, who can read them via their clients
c) Unauthorized users cannot access other users' file contents

3.4.2 Building blocks

1. SQL Queries: Verify ownership and sharing permissions.
2. Database Tables: `Comp3334_AccFile` table stores file ownership (accNo, fileNo, fileName, fileData), and Comp3334_ShareFile manages sharing (fileNo, accNo).

3.4.3 Ownership Check

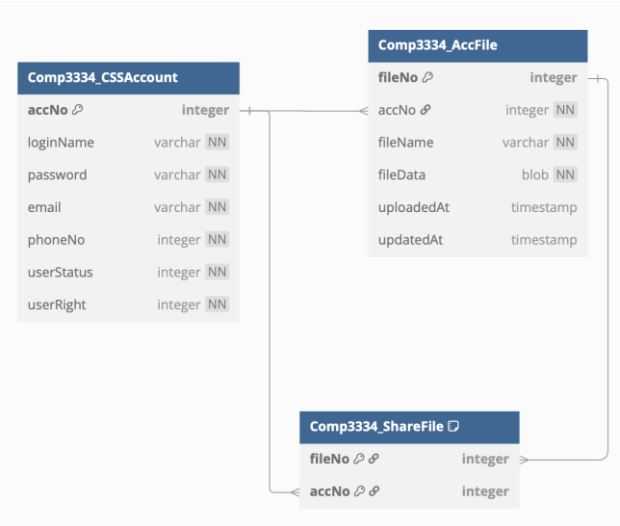
Before user proceeds to share/unshare/edit/delete operations of target file, it will attempt retrieve the record in Comp3334_AccFile table to check if target file is owned by the user (See SQL query below):

```
SELECT accNo FROM Comp3334_AccFile WHERE fileNo = ? AND accNo = ?
```

3.4.4 Sharing mechanism

Comp3334_ShareFile table (See below Figure)'s primary key constructed from two foreign keys:

1. fileNo (referenced from `Comp3334_AccFile` table)
2. accNo (referenced from `Comp3334_CSSAccount` table)



To share a file with designated user,

1. File is checked to be owned by owner (Refer to 3.3.3)
2. Owner file's number as fileNo and designated user account's number as accNo are inserted to `Comp3334_ShareFile` table within Oracle database (See SQL statement below):

```
// Share the file
sql = "INSERT INTO Comp3334_ShareFile (fileNo, accNo) SELECT ?, accNo FROM Comp3334_CSSAccount WHERE loginName = ?";
```

3. Shared user can download the file shared with them:

```
================================================================
COMPUTER SYSTEMS SECURITY (Comp3334_CSS)
Date: 2025-04-10 13:19:06.573

AccNo:3 | Name:user3 | Email:user3@connect.polyu | PhoneNo:45678921
================================================================
** Command List **
[0]:Logout & Exit          [1]:Upload File      [2]:Download File
[3]:ReUpload File(Edit)    [4]:File ShareTo     [5]:File UnshareTo
[6]:Delete File            [7]:Reset PWD

** Files List: **

[Index:01] [Owner:yyy] [File:abcd.txt]
fileList:[1], fileList[0]:1

Enter CMD:
```

First, shared file is displayed in the designated user's command interface retrieved with SQL statement below.

```
// --start of displaying the share file by other user--
sql = "SELECT a.fileNo, c.loginName, c.email, a.fileName " +
        "FROM Comp3334_AccFile a " +
        "JOIN Comp3334_ShareFile s ON a.fileNo = s.fileNo " +
        "JOIN Comp3334_CSSAccount c ON a.accNo = c.accNo " +
        "WHERE s.accNo = ?";
```

Before user decrypts and downloads target file, it will attempt retrieve the record in `Comp3334_AccFile` table to check if the target file is owned by user or is shared with the user (See SQL query below):

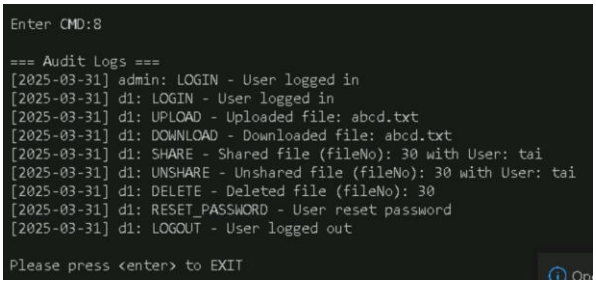
```
SELECT fileName, fileData FROM Comp3334_AccFile WHERE fileNo = ? AND (accNo = ? OR fileNo IN
(SELECT fileNo FROM Comp3334_ShareFile WHERE accNo = ?))
```

4. Only the owner of the file is allowed to unshare the file

3.5 Log Auditing:

3.5.1 Theoretical Design

The records of audit logs include 3 types of information from user operations: timestamp, username and user actions, detailing specific file or user that is traceable by the administrator. The following operations is recorded by logs: LOGIN, LOGOUT, UPLOAD, DOWNLOAD, SHARE, UNSHARE, DELETE, RESET_PASSWORD.



Log information is stored in the `COMP3334_LogCSS` table within Oracle database (see its fields in SetUpDB_SQL.docx). There are two main parts to deliver the function:

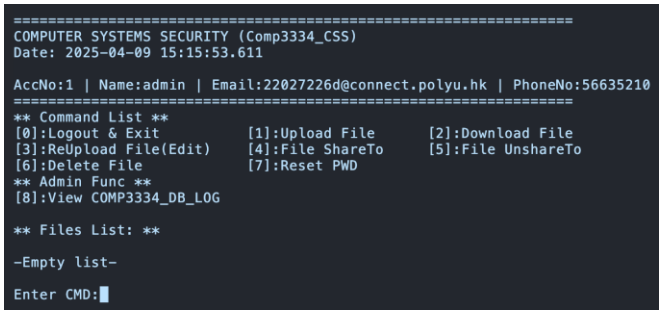
3.5.2 Log Insertion

To insert log every critical user operation, it executes SQL statement INSERT into the COMP3334_LogCSS table with 3 parameters:

1. accNo,
2. formattedDate (Local Date formatted by `java.time.format.DateTimeFormatter`),
3. action

3.5.3 Log Retrieval

1. Only admin account's command interface displays the option ' [8]: View COMP3334_DB_LOG '



2. To retrieve log info every critical operation, it executes SELECT SQL statement from the `COMP3334_LogCSS` table to retrieve accNo, action, (Date object) logDate.

3. it executes SELECT SQL statement from the `COMP3334_CSSAccount` table to retrieve loginName from accNo
4. To make Date object printable, `java.text.SimpleDateFormat` is utilized to format logDate
5. Printed Log format: "`[%logDate%] %loginName% : %action% `"

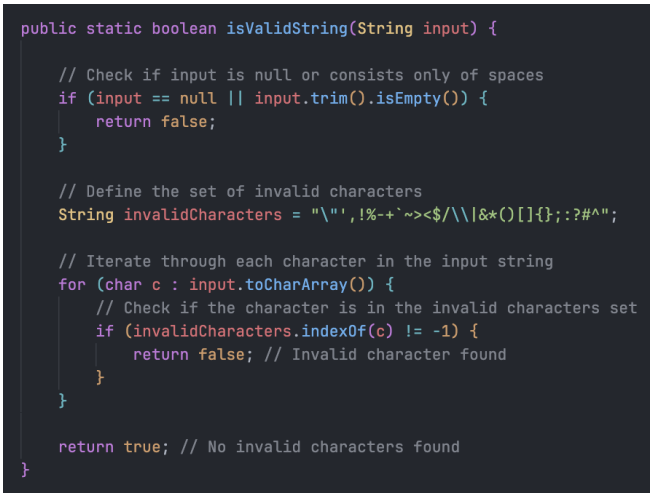
3.6 General Security Protection:

3.6.1 Theoretical design

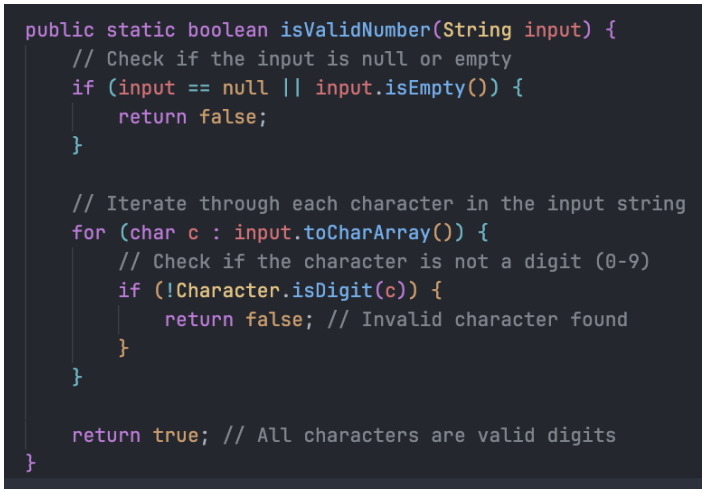
General security protection targets vulnerabilities like directory traversal attacks (via invalid file names) and SQL injections. Therefore, the design focuses on input validation and secure query execution to safeguard the system:

1. Input validation: checks file names and user inputs for malicious patterns
2. Prepared Statements: prevents SQL injection by passing queries as parameters

3.6.2 Input validation



```
public static boolean isValidString(String input) {

    // Check if input is null or consists only of spaces
    if (input == null || input.trim().isEmpty()) {
        return false;
    }

    // Define the set of invalid characters
    String invalidCharacters = "\"',!%-+`~><$/\\|&*()[]{};:?#^";

    // Iterate through each character in the input string
    for (char c : input.toCharArray()) {
        // Check if the character is in the invalid characters set
        if (invalidCharacters.indexOf(c) != -1) {
            return false; // Invalid character found
        }
    }

    return true; // No invalid characters found
}
```



```
public static boolean isValidNumber(String input) {
    // Check if the input is null or empty
    if (input == null || input.isEmpty()) {
        return false;
    }

    // Iterate through each character in the input string
    for (char c : input.toCharArray()) {
        // Check if the character is not a digit (0-9)
        if (!Character.isDigit(c)) {
            return false; // Invalid character found
        }
    }

    return true; // All characters are valid digits
}
```

Figure A **Figure B**

1. **Figure A** checks through the string input with the pattern for invalid characters: `\,!%-+`~><$/\\|&*()[]{};:?#^`
2. **Figure B** checks through and ensures the string input is a digit
3. The system is expecting user to input file name only, inputting "`../test.txt`" includes invalid "/" therefore cannot proceed the operation

See below the codes for an example input checking:

```
System.out.print(s:"*Please ensure the file is store in 'uploads' folder*\n*E.g: abcd.txt*\nEnter the file name to upload to DB : ");
String filename = console.readLine();
while (!isValidString(filename)) {
    System.out.print(s:"\nCan't including any invalid symbol like \"',!%-+`~><$/\\|&*()[]{};:?#^\nEnter the file name to upload to DB :");
    filename = console.readLine();
}
```

3.6.3 Prepared Statements

All database queries use parameterized `PreparedStatement` objects to block malicious SQL execution

1. Make use of "?" (hidden parameters) to insert values into database tables

```
sql = "INSERT INTO Comp3334_AccFile (accNo, fileName, fileData) VALUES (?, ?, ?)";
```

2. Server sets the parameters received from the client via network

```
try {
    PreparedStatement pstmt = conn.prepareStatement(sql);
    pstmt.setInt(parameterIndex:1, accNo);
    pstmt.setString(parameterIndex:2, fileName);
    pstmt.setBytes(parameterIndex:3, encryptedFileData);
    rows = pstmt.executeUpdate();
    if(rows>0) rs = true;

} catch (Exception e) {
    e.printStackTrace();
```

3.7 EXTENDED functionalities:

3.7.1 Multi-Factor Authentication (MFA): TOTP

```
pwd = storedhash;

String[] parts = storedhash.split(regex:":");
// System.out.println("\nT:"+TOTP_secret+":PK"); //COMP3334
TOTP_secret += toTOTPKEY(parts[1]); // client TOTP key = individual SK
// System.out.println("\nT:"+TOTP_secret+":SK"); //COMP3334 + hash
```

For the TOTP function, the system will use the user's hash as the key(secret) of a part for the TOTP_secret:

```
TOP_secret = "COMP3334" + <user's hash>
```

and the generated TOTP codes will be stored into 'phone.txt'. To enhance security, these codes is set to expire every 30 seconds. The 30-second window can ensure that codes expire quickly, and reduce the risk of interception or replay attacks (M'Raihi et al., 2011). If a user attempts to use another user's TOTP code or enters a code outside the window, verification would not be passed because of mismatch.

3.7.2 Other security designs: Account lock/unlock by admin

When user fails TOTP verification but successfully passes the hashed password check, the database log will record such as:

```
LOGIN - Fail with TOTP (Curr: Activate Account) OR

LOGIN - Fail with TOTP (Curr: Locked Account)
```

It allows the admin to review suspicious activity, the admin can then update the user's status from 0 to 1 to lock the account if needed. For locked accounts, users will only see a message stating, “Your account has been locked!” They will have no permissions to perform any actions and must contact the admin to request unlock account.

4. Test Cases

4.1 Test Case 1: Unauthorized Modification or Deletion of Shared Files

Test Objective

This test verifies that a user who has been shared a file by another user cannot modify or delete that file, ensuring that only the file owner retains edit and delete privileges.

Test Steps

1. **User Setup**:
    a. Create two accounts: User A (yyy) as the file owner and User B (yyy2) as the recipient of the shared file.
2. **File Upload and Sharing**:
    a. User A logs in via Client.java, uploads a file (testfile.txt) using command [1]: Upload File, which is encrypted with AES-256 and stored in Comp3334_AccFile.
    b. User A shares the file with User B using command [4]: File ShareTo, specifying User B's username (yyy2). The server executes:

INSERT INTO Comp3334_ShareFile (fileNo, accNo)

SELECT ?, accNo FROM Comp3334_CSSAccount WHERE loginName = ?

3. **Unauthorized Modification Attempt**:
    a. User B logs in, views the shared file in the file list, and attempts to edit it using command [3]: ReUpload File (Edit) by entering the file's index.
    b. The server checks ownership with:

SELECT accNo, fileName, fileData FROM Comp3334_AccFile WHERE fileNo = ? AND accNo = ?

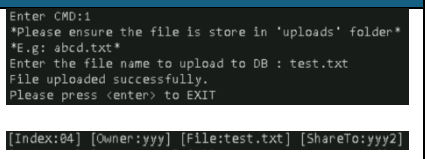
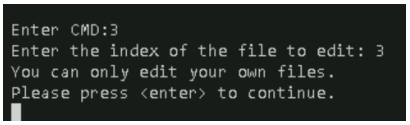
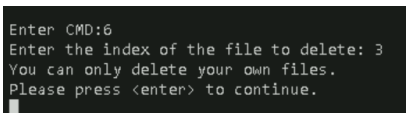
4. **Unauthorized Deletion Attempt**:
    a. User B attempts to delete the file using command [6]: Delete File by entering the file's index.

b. The server checks ownership with:

SELECT accNo FROM Comp3334_AccFile WHERE fileNo = ? AND accNo = ?

5. Verification:

Confirm User B receives an error for both attempts.

Step	Action	Result
Upload and Share	User A uploads and shares file	 Pass
Edit Attempt	User B tries to edit shared file	 Pass
Delete Attempt	User B tries to delete shared file	 Pass

Security Measures

- **Ownership Check**: SQL queries in selectEditFile and checkFileOwner restrict edit/delete actions to the file owner.
- **Privilege Separation**: Sharing grants read-only access, while edit/delete rights remain with the owner.

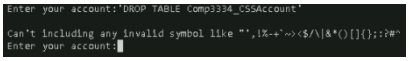
4.2 Test Case 2: SQL Injection Attack

Test Objective

This test ensures the system is resilient to SQL injection attacks, preventing malicious input from bypassing authentication or exposing sensitive data.

Test Steps

1. **SQL Injection Attempt**: In the login interface of Client.java, enter a malicious SQL statement in the username field (' OR '1'='1).

Step	Action	Result
SQL Injection Attempt	Input 'DROP TABLE Comp3334_CSSAccount' at Enter your account.	 Pass

Security Measures

- **Parameterized Queries**: The system uses PreparedStatement objects (pstmt.setString(1, acc_name)) for all SQL queries, preventing injection by treating input as data rather than executable code.
- **Input Validation**: The isValidString method filters special characters from user inputs, further reducing injection risks.

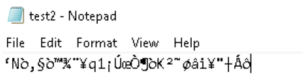
4.3 Test Case 3: Unauthorized Access to Secret Keys

Test Objective

Verify that unauthorized users cannot obtain the secret key (encryption_key.key) used for file encryption, ensuring the confidentiality of encrypted files.

Test Steps

1. **User Setup**:
    a. Create two accounts: User A (yyy) as the file owner and User B (yyy2) as an unauthorized user.
2. **File Upload and Key Generation**:
    a. User A logs in via Client.java, uploads a file (test2.txt) using command [1]: Upload File.
    b. During this process, loadOrGenerateKey generates or loads the AES-256 secret key, storing it in encryption_key.key in User A's local directory.
3. **Unauthorized Access Attempt via System**:
    a. User B logs in on a separate instance of Client.java.
    b. User B attempts to access User A's uploaded file (test2.txt) using command [2]: Download File after User A shares it with User B via [4]: File ShareTo.
    c. Check if User B's client can access or infer the secret key from the downloaded encrypted data.
4. **Verification**:
    a. Confirm User B cannot decrypt User A's file without the key.

Step	Action	Result
Read file without "encryption_key.key"	User B tried to read User A file without "**encryption_key.key**"	 Pass

Security Measures

- **Client-Side Encryption**: Files are encrypted/decrypted on the client side using AES-256, with the key never transmitted to the server.

- **Key Storage**: The secret key is stored locally in encryption_key.key, reducing server-side exposure but relying on client-side security.

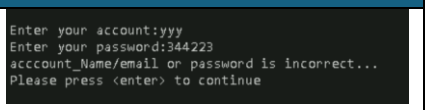
4.4 Test Case 4: Unauthorized User Exploiting TOTP

Test Objective

Verify that an unauthorized user cannot bypass authentication using stolen TOTP.

Test Steps

1. User A (yyy) logs in, generating TOTP in phone.txt.
2. Someone steals TOTP from phone.txt and attempts login.
3. Verify login outcomes.

Step	Action	Result
Login Attempt	**Someone steals TOTP from phone.txt and attempts login.**	 Pass

Security Measures

- **Multi-Factor Authentication (MFA)**: The system employs Multi-Factor Authentication (MFA), combining TOTP as a second authentication factor with traditional password verification.
- **Protection**: Even if an unauthorized user steals the TOTP from phone.txt, they cannot log in without User A's password. This dual requirement ensures security remains intact even if one credential is compromised.

5. Future Works

While our secure online storage system delivered the required core and extended functionalities, below are the areas for future enhancements:

5.1 Advanced Key Handling

Current Limitation

If the client machine is compromised, an active adversary could use the key to decrypt all files.

Future enhancemen:

We will implement per-user keys isolating each user's data to increase complexity for adversary not decrypting files easily, such as storing user key files with encrypted file names without telling information about the user.

5.2 File Versioning

Current Limitation

The system does not support file versioning, which may risk data loss from accidental overwrites, deletions, or ransomware attacks.

Future enhancement

We will implement file versioning to store historical copies of files on the server as a form of encrypted backups. To enhance data resilience under or after attacks, users can revert to previous versions, providing them with recovery options.

5.3 Advanced Access Control

Current Limitation

The system implements basic ownership-based access control but it does not support more advanced cases like fine-grained permissions or role-based access control (RBAC), which is booster for scalability and efficiency in permission granting (Sandhu, 1998).

Future enhancement

We will enhance by implement RBAC, admins can grant specific permissions on specific roles, such as Admin role: can read, write, share and delete the files, Visitor role: read-only, Editor role: read-and-write only, improving security in collaborative use cases.

6. Reference

Oracle. (n.d.). Java Cryptography Architecture (JCA) Reference Guide for Java Platform Standard Edition 6. Retrieved April 13, 2025, from https://docs.oracle.com/javase/6/docs/technotes/guides/security/crypto/CryptoSpec.html#trans

Dworkin, M. (2007) Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)and GMAC National Institute of Standards and Technology. https://csrc.nist.gov/projects/block-cipher-techniques/bcm

Kaliski, B. (2000). PKCS #5: Password-based cryptography specification version 2.0 (RFC 2898). Internet Engineering Task Force. https://tools.ietf.org/html/rfc2898

Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (2018). Handbook of applied cryptography. CRC Press. (Original work published 1996) https://doi.org/10.1201/9780429466335

Sandhu, R. S. (1998). Role-Based Access Control. Advances in Computers, 46, 237–286. https://doi.org/10.1016/S0065-2458(08)60206-5

M'Raihi, D., Rydell, J., Pei, M., & Machani, S. (2011). TOTP: Time-based one-time password algorithm (RFC 6238). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc6238